```
void ddpoly(VecDoub_I &c, const Doub x, VecDoub_O &pd)                    poly.h
```
Given the coefficients of a polynomial of degree nc as an array c[0..nc] of size nc+1 (with c[0] being the constant term), and given a value x, this routine fills an output array pd of size nd+1 with the value of the polynomial evaluated at x in pd[0], and the first nd derivatives at x in pd[1..nd].

```
{
    Int nnd,j,i,nc=c.size()-1,nd=pd.size()-1;
    Doub cnst=1.0;
    pd[0]=c[nc];
    for (j=1;j<nd+1;j++) pd[j]=0.0;
    for (i=nc-1;i>=0;i--) {
        nnd=(nd < (nc-i) ? nd : nc-i);
        for (j=nnd;j>0;j--) pd[j]=pd[j]*x+pd[j-1];
        pd[0]=pd[0]*x+c[i];
    }
    for (i=2;i<nd+1;i++) {          After the first derivative, factorial constants come in.
        cnst *= i;
        pd[i] *= cnst;
    }
}
```

As a curiosity, you might be interested to know that polynomials of degree $n > 3$ can be evaluated in *fewer* than n multiplications, at least if you are willing to precompute some auxiliary coefficients and, in some cases, do some extra addition. For example, the polynomial

$$P(x) = a_0 + a_1 x + a_2 x^2 + a_3 x^3 + a_4 x^4 \tag{5.1.1}$$

where $a_4 > 0$, can be evaluated with three multiplications and five additions as follows:

$$P(x) = [(Ax + B)^2 + Ax + C][(Ax + B)^2 + D] + E \tag{5.1.2}$$

where A, B, C, D, and E are to be precomputed by

$$\begin{aligned} A &= (a_4)^{1/4} \\ B &= \frac{a_3 - A^3}{4A^3} \\ D &= 3B^2 + 8B^3 + \frac{a_1 A - 2a_2 B}{A^2} \\ C &= \frac{a_2}{A^2} - 2B - 6B^2 - D \\ E &= a_0 - B^4 - B^2(C + D) - CD \end{aligned} \tag{5.1.3}$$

Fifth-degree polynomials can be evaluated in four multiplies and five adds; sixth-degree polynomials can be evaluated in four multiplies and seven adds; if any of this strikes you as interesting, consult references [3-5]. The subject has something of the same flavor as that of fast matrix multiplication, discussed in §2.11.

Turn now to algebraic manipulations. You multiply a polynomial of degree $n - 1$ (array of range [0..n-1]) by a monomial factor $x - a$ by a bit of code like the following,

```
c[n]=c[n-1];
for (j=n-1;j>=1;j--) c[j]=c[j-1]-c[j]*a;
c[0] *= (-a);
```

Likewise, you divide a polynomial of degree n by a monomial factor $x - a$ (synthetic division again) using

```
rem=c[n];
c[n]=0.;
for(i=n-1;i>=0;i--) {
    swap=c[i];
    c[i]=rem;
    rem=swap+rem*a;
}
```

which leaves you with a new polynomial array and a numerical remainder rem.

Multiplication of two general polynomials involves straightforward summing of the products, each involving one coefficient from each polynomial. Division of two general polynomials, while it can be done awkwardly in the fashion taught using pencil and paper, is susceptible to a good deal of streamlining. Witness the following routine based on the algorithm in [3].

poly.h

```
void poldiv(VecDoub_I &u, VecDoub_I &v, VecDoub_O &q, VecDoub_O &r)
```

Divide a polynomial u by a polynomial v, and return the quotient and remainder polynomials in q and r, respectively. The four polynomials are represented as vectors of coefficients, each starting with the constant term. There is no restriction on the relative lengths of u and v, and either may have trailing zeros (represent a lower degree polynomial than its length allows). q and r are returned with the size of u, but will usually have trailing zeros.

```
{
    Int k,j,n=u.size()-1,nv=v.size()-1;
    while (nv >= 0 && v[nv] == 0.) nv--;
    if (nv < 0) throw("poldiv divide by zero polynomial");
    r = u;                                  May do a resize.
    q.assign(u.size(),0.);                  May do a resize.
    for (k=n-nv;k>=0;k--) {
        q[k]=r[nv+k]/v[nv];
        for (j=nv+k-1;j>=k;j--) r[j] -= q[k]*v[j-k];
    }
    for (j=nv;j<=n;j++) r[j]=0.0;
}
```

5.1.1 Rational Functions

You evaluate a rational function like

$$R(x) = \frac{P_\mu(x)}{Q_\nu(x)} = \frac{p_0 + p_1 x + \cdots + p_\mu x^\mu}{q_0 + q_1 x + \cdots + q_\nu x^\nu} \tag{5.1.4}$$

in the obvious way, namely as two separate polynomials followed by a divide. As a matter of convention one usually chooses $q_0 = 1$, obtained by dividing the numerator and denominator by any other q_0. In that case, it is often convenient to have both sets of coefficients, omitting q_0, stored in a single array, in the order

$$(p_0, p_1, \ldots, p_\mu, q_1, \ldots, q_\nu) \tag{5.1.5}$$

The following object encapsulates a rational function. It provides constructors from either separate numerator and denominator polynomials, or a single array like (5.1.5) with explicit values for n $= \mu + 1$ and d $= \nu + 1$. The evaluation function makes Ratfn a functor, like Poly. We'll make use of this object in §5.12 and §5.13.